# automx2

***Email client configuration made easy***

Version 2026.2, 2026-04-28

# 1. Copyright

automx2 is Copyright © 2019-2026 Ralph Seichter. automx2 is licensed under the GNU General Public License V3 or later. The project is hosted on GitHub in the rseichter/automx2 [https://github.com/rseichter/automx2] repository.

# 2. Contact

Should you be interested in supporting the project as a sponsor, you can find a contact email address in the sponsorship section.

# 3. Preface

This document explains how automx2 works, how automated mail client configuration works, and what it takes to install and configure automx2. If you are already familiar with automated mailbox configuration methods you may want skip the following sections and jump right ahead to Installing automx2 and Configuring automx2.

# 4. How does automx2 operate?

automx2 is a web service. It is usually located behind a web server like NGINX and waits for configuration requests. When a mail user agent (MUA), a.k.a. mail client, requests configuration it contacts the web server. The web server then acts as a proxy and forwards all requests to automx2 and passes answers back to the MUA.

# 5. How does auto config work?

Modern email clients (Mail User Agents) can look for configuration data when a user begins to create a new account. They will either send the user's mail address to a service and ask the service to reply with configuration that suits the user's profile, or they will query the DNS system for advice.

Using a specialized mail account configuration service allows for individualized setups. It also allows to enforce a specific policy, which for example configures the mail client to use a specific authentication mechanism. Quering the DNS for mail service locations allows for generic instructions, but it doesn't give as much control over settings as a specialized service like automx2 will do.

As of today, there are four methods that help configuring a mail account. Three of them – Autoconfig, Autodiscover and Mobileconfig – have been developed by vendors to cover their products' spe-

cific needs. The fourth is an RFC standard specifying the aformentioned more general DNS SRV resource records method.

The vendor specific methods have in common that the mail client seeking configuration needs to send a request, which includes at least the user's mail address, to a configuration service. The service will use the mail address to lookup configuration data and will return that data as response to the client. Format – XML response or file – and complexity differ depending on the method.

automx2 implements everything necessary to configure email accounts. Functionality to configure calendar or address book settings is not included. This may change in some future version, but the focus of automx2 is email.

## 5.1. Autoconfig

Autoconfig is a proprietary method developed by the Mozilla foundation. It was designed to configure a mail account within Thunderbird, and other email suites like Evolution and KMail have adopted [https://wiki.mozilla.org/Thunderbird:Autoconfiguration:ConfigFileFormat] the mechanism.

When a user begins to create a new mail account she is asked to enter her realname and mail address, e.g. *alice@example.com*. Thunderbird will then extract the domainpart (*example.com*) from the mail address and build a list of URIs to search for a configuration web service in the following order:

```
https://autoconfig.thunderbird.net/v1.1/example.com
https://autoconfig.example.com/mail/config-v1.1.xml?emailaddress=alice@example.com
https://example.com/.well-known/autoconfig/mail/config-v1.1.xml
http://autoconfig.thunderbird.net/v1.1/example.com
http://autoconfig.example.com/mail/config-v1.1.xml?emailaddress=alice@example.com
http://example.com/.well-known/autoconfig/mail/config-v1.1.xml
```

A configuration service such as automx2 listening on one of the listed URIs will receive the request, process it and respond with a set of configuration instructions.

Thunderbird will use the instructions to automatically fill in the required fields in the account. The only remaining task for the user is to confirm the settings. After that she can immediately start to use her new mail account.

## 5.2. Autodiscover

Autodiscover is a proprietary method developed by Microsoft. The protocol version supported by automx2 was designed to configure a mail account within Outlook 2016 ff. Service lookups use the URLs shown below and, as a fallback option, DNS lookups. Please note that Microsft uses a different autodiscover mechanism for Office 365, which is not yet supported by automx2 because information about the technical details are not available free of charge.

```
https://example.com/autodiscover/autodiscover.xml
https://autodiscover.example.com/autodiscover/autodiscover.xml
```

```
http://autodiscover.example.com/autodiscover/autodiscover.xml

dns: autodiscover.example.com
dns: _autodiscover._tcp.example.com
```

All HTTP(S) queries send a POST request and submit XML which contains information about the account that should be configured. The DNS queries search for a `CNAME` resource record first, which is supposed to redirect the mail client to a resource outside the mailbox owners domain, e.g. *alice@example.com* would be redirected to `service.example-provider.com` for configuration instructions. If the first DNS query fails the client may be redirected to a configuration service using a `SRV` RR like this:

```
_autodiscover._tcp.example.com.  0   443 service.example-provider.com.
```

The `SRV` RR used in the example above would send Alice's client to `service.example-provider.com` and tell it to send the query to the configuration service on port `443`.

## 5.3. Mobileconfig

Requests and responses use Apple-proprietary content types with an underlying property list [https://en.wikipedia.org/wiki/Property_list] format, for both iOS and macOS. The easiest method to configure Apple device using automx2 is to visit your mobileconfig URL using the Safari browser directly on the device itself. To reduce the risk of man-in-the-middle attacks, please establish an encrypted HTTPS connection for this step. Do not use unencrypted connections unless you are on your local network. The appropriate email address needs to be passed via URL parameter, using `%40` to encode the @ symbol.

```
https://autoconfig.example.com/mobileconfig/?emailaddress=alice%40example.com
```

Save and then install the offered configuration profile on your devices. The device OS will warn you about the profile being *unsigned* (meaning it does not come with a cryptographical signature). This is expected behaviour. Make sure the configuration profile was obtained from either your personal server or from a trusted third party, by way of examining the HTTPS certificate and URL.

## 5.4. DNS SRV resource records

```
_imap._tcp.example.com          SRV  10  20  143  mail.example.com.
_imaps._tcp.example.com         SRV  0   1   993  .
_pop3._tcp.example.com          SRV  0   1   110  .
_pop3s._tcp.example.com         SRV  0   1   995  .
_smtp._tcp.example.com.         SRV  0   1   25   .
_submission._tcp.example.com.   SRV  10  20  587  mail.example.com.
```

# 6. Installing automx2

automx2 requires Python version 3.7 or greater, ideally in the form of a virtual Python environment, to run. Check the python3 version like this:

```
$ python3 --version
Python 3.12.3
```

> *Don't run as root*
>
> If you use a port number greater than 1024 (I suggest port 4243), the application does not require superuser privileges when running. Doing so would pose a security risk and is therefore strongly discouraged. I recommend creating a fresh user account called `automx2`.

## 6.1. Package based installation

There are packages available for Arch Linux [https://aur.archlinux.org/packages/automx2] and NixOS [https://search.nixos.org/packages?type=packages&query=automx2]. Note that these packages may not alway be up-to-date, because Linux distributions have their own release schedules. Please make sure to check the version numbers if you decide to go this route.

## 6.2. Pip based installation

This is the method which should always provide you with the latest release. The path `/srv/www/automx2` will be used as an example throughout this documentation. The BASH shell commands below should work with any modern Linux distribution.

```
# Best practice: Create a fresh user account.
sudo useradd --home-dir /srv/www/automx2 --create-home automx2

# Alternative: If the user account already exists.
# sudo bash -c 'mkdir -p /srv/www/automx2 && chown automx2 /srv/www/automx2'
```

Next, make sure to either login as the user created above, or change to this user via the 'su' command. This is important to ensure the correct file permissions. Now download the script that will install your automx2 service. Installation requires BASH plus either curl or wget to download additional data. The script will abort if neither of the latter two can be found.

```
cd /srv/www/automx2
wget https://raw.githubusercontent.com/rseichter/automx2/master/contrib/install.sh
# Alternative: curl -O
https://raw.githubusercontent.com/rseichter/automx2/master/contrib/install.sh
bash install.sh
```

Executing the setup script will create a Python virtual environment called `.venv` in the current directory. You may pass a custom directory path as an argument to install.sh, if necessary. To ensure a clean slate, installation will abort if the destination path already exists.

```
# Example of how to pass a custom directory
bash install.sh /path/to/your/venv
```

The script will automatically download the automx2 Python module and place it inside the newly created virtual environment. It will also create a launch script `.venv/bin/flask.sh`, which can run automx2 after you prepared the configuration.

## 6.3. Updating

If you use pre-built packages, please consult the documentation for your specific Linux distribution. The following applies only to pip based installations:

Change to the directory where automx2 has been installed previously. Activate the virtual environment as usual and use pip's `--upgrade` option:

```
cd /srv/www/automx2
.venv/bin/pip install --upgrade automx2
```

# 7. Configuring automx2

automx2 uses a file to read runtime instructions from and a database to lookup mail account configuration data.

## 7.1. Placeholders

To make configuration more convenient, automx2 supports Mozilla-style placeholders [https://wiki.mozilla.org/Thunderbird:Autoconfiguration:ConfigFileFormat#Placeholders]. For example, the string `%EMAILADDRESS%` in database records will be replaced with the email address specified during the query. While based on a proprietary feature of Autoconfig, automx2 also applies placeholders to Autodiscover and Mobileconfig responses.

## 7.2. Runtime configuration

The configuration file defines automx2 runtime behaviour, and it specifies the backend automx2 should read mailbox account configuration data from.

> *Running without runtime config*
>
> If you launch automx2 without a configuration file, it will use internal defaults. These are suitable for testing only. Launched without a config it will use an in-memory SQLite database and all data will be lost once the application terminates.

During startup automx2 searches for runtime configuration instructions in the following locations. The first match will determine the configuration used.

```
env  : AUTOMX2_CONF  ①
file : ~/.automx2.conf
file : /etc/automx2/automx2.conf
file : /etc/automx2.conf
```

① If present, the environment variable AUTOMX2_CONF must point to the absolute path of a configuration file.

To specify parameters and options automx2 uses an INI file [https://docs.python.org/3.9/library/configparser.html#supported-ini-file-structure] syntax. The example configuration [https://github.com/rseichter/automx2/blob/master/contrib/automx2-sample.conf] that ships with automx2 looks like this:

```
[automx2]
# A typical production setup would use loglevel WARNING.
loglevel = DEBUG
# Echo SQL commands into log? Used for debugging.
db_echo = no
# In-memory SQLite database
db_uri = sqlite:///:memory:

# SQLite database in a UNIX-like file system
#db_uri = sqlite:////var/lib/automx2/db.sqlite

# MySQL database on a remote server. This example does not use an encrypted
# connection and is therefore *not* recommended for production use.
#db_uri = mysql://username:password@server.example.com/db

# Number of proxy servers between automx2 and the client (default: 0).
# If your logs only show 127.0.0.1 or ::1 as the source IP for incoming
# connections, proxy_count probably needs to be changed.
proxy_count = 1
```

Place the content of the example configuration into one of the configuration locations automx2 looks for and adapt it to your needs. Then configure the database backend with data that suits your setup, as described below.

## 7.3. Testing standalone automx2

If you want to verify a vanilla installation of automx2 works, you can populate it with internal test data. Start automx2 as described in section Running automx2 and send the following request to populate your database:

```
curl http://127.0.0.1:4243/initdb/
```

This example assumes you are running automx2 on localhost listening on TCP port 4243, which is the suggested default port.

Once you have populated the database with sample data you can test if automx2 works. Use curl to send an account configuration request for user@example.com:

```
curl 'http://127.0.0.1:4243/mail/config-
v1.1.xml?emailaddress=alice@example.com&name=Alice%20Jones'
```

As shown in the example, make sure to quote the URL as necessary. Otherwise, your command shell might perform pattern matching for characters like the question mark ? (FISH does).

## 7.4. Password support

For a long time I have been averse to supporting passwords in the generated configuration data, because of the risks involved. By the very nature of the process, *cleartext* passwords are required. This means that passwords can potentially be intercepted in transit, e.g. between a HTTPS proxy and automx2. Even worse, should the receiving user decide to save a configuration file locally, that file will contain the user's password in plain text, visible for everybody with read access to the file. Also, depending on how automx2 is configured, cleartext passwords can potentially be written to log files.

Alas, my reluctance to implement password support has resulted in some code forks where people added the feature anyway. To mitigate this, I decided to provide a limited implementation. automx2 version 2024.2 introduced password support for Apple's Mobileconfig. iOS devices download configuration profiles into a staging area by default, not offering the user an easy method to save the data elsewhere.

The password support feature is deliberately disabled by default. To opt in, you need to set the environment variable `PERMIT_CLEARTEXT_PASSWORDS=I_understand_the_risks`.

> *Your responsibility*
>
> Caveat emptor: By opting in to this feature, you acknowledge that you take responsibility for any possible harm that could result from leaked passwords. Please ask yourself if a small amount of added convenience for users, i.e. having to enter their passwords less often during infrequently occurring device setups, is worth the risks.

Once the feature is activated, you can send GET requests with additional parameters. Sending a password with support disabled will result in an error log message, and the supplied password will be ignored. URL encoding is very important here. I suggest that you create a HTML landing page for your users, with explanation about the risks inolved. Include a `<form>` to submit the necessary fields to automx2.

```
curl
'http://127.0.0.1:4243/mobileconfig?emailaddress=jd@example.com&name=John%20Doe&passwo
```

```
rd=ThisIsInsecure'
```

## 7.5. Database configuration

automx2 uses the SQLAlchemy toolkit to access databases. This allows a variety of databases, a.k.a. dialects [https://docs.sqlalchemy.org/en/latest/dialects/], to be used, simply by defining the appropriate connection URL.

> *API based configuration*
>
> I consider adding an API for configuration changes in an upcoming version but have not decided when that might happen. Feel free to contact me if you are interested in a sponsorship.

### 7.5.1. Database support

While you probably already have SQLite support available on your local machine, you may need to install additional Python packages for PostgreSQL, MySQL, etc. Detailed instructions to support a particular database dialect are out of scope for this document. Please search the Internet for detailed instructions on supporting a particular dialect. The SQLAlchemy documentation provides a useful starting point.

Instead of manually setting up tables, I recommend using the built-in method to create the necessary DB structure by sending an HTTP GET request to the `/initdb/` service endpoint. This will also populate the database with some predefined example data. Alternatively, you can send a POST request with custom JSON data to the same endpoint, as described below.

> *Purging the database*
>
> Sending an HTTP DELETE request to `/initdb/` will purge all existing data. Be sure to limit access to this service route accordingly!

Note that support for automatic, Alembic based database migration was removed in automx2 version 2026.2. DB changes were too infrequent to warrant the third-party dependency.

### 7.5.2. SQLite

This section demonstrates what you need to do to in order to use SQLite version 3 or higher as a backend database for automx2.

Step 1: Set the database URI in your automx2 configuration. Please note that specifying an absolute path for the database requires a total of four slashes after the schema identifier:

```
[automx2]
db_uri = sqlite:////var/lib/automx2/db.sqlite
```

Step 2: Launch automx2 and access the DB initialisation URL.

```
# Method 1: Populate DB with example data
curl -X GET http://127.0.0.1:4243/initdb/
# Method 2: Populate DB based on the content of a JSON file
curl -X POST --json @mydata.json http://127.0.0.1:4243/initdb/
```

### 7.5.3. JSON

Starting with automx2 version 2022.0, JSON data can be used to populate the database in a simplified manner, without the need to use SQL statements. JSON based configuration is meant to ease setup for small automx2 installations and makes certain assumptions about the relationships between servers and domains. If you require more flexibility, I recommend that you manually create the appropriate database records.

Supported JSON data formats are shown here. Note that the format was changed to version 2 in automx2 release 2026.1, which added support for LDAP servers during seeding.

Current V2 format [https://github.com/rseichter/automx2/blob/master/contrib/seed-sample.json], for release 2026.1:

```
{
  "_comment": "For automx2 release 2026.1",
  "version": 2,
  "provider": "Example Inc.",
  "domains": [
      { "name": "example.com" },
      { "name": "example.net" },
      {
        "name": "example.org",
        "ldapserver_id": 42
      }
    ],
  "ldapservers": [
      {
        "id": 42,
        "name": "ldap.example.com",
        "port": 636,
        "use_ssl": "yes",
        "search_base": "ou=people,dc=example,dc=com",
        "search_filter": "(mail={0})",
        "attr_uid": "uid",
        "attr_cn": "cn",
        "bind_password": "CLEARTEXT",
        "bind_user": "uid=techuser,dc=example,dc=com"
      },
      {
        "id": 99,
        "name": "unused.example.net",
        "port": 389,
        "search_base": "dc=example,dc=net",
```

```
        "bind_password": "UNSAFE",
        "bind_user": "uid=foo,dc=example,dc=net"
      }
    ],
  "servers": [
      {
        "type": "imap",
        "name": "imap.example.com"
      },
      {
        "type": "smtp",
        "name": "smtp.example.com"
      },
      {
        "type": "caldav",
        "port": 443,
        "url": "https://www.example.net/SOGo/dav/%EMAILADDRESS%/Calendar/personal/"
      },
      {
        "type": "carddav",
        "port": 443,
        "url": "https://www.example.net/SOGo/dav/%EMAILADDRESS%/Contacts/personal/"
      }
    ]
}
```

Older V1 format [https://github.com/rseichter/automx2/blob/master/contrib/seed-sample-old.json], only for releases 2026.0 and earlier:

```
{
  "_comment": "For automx2 release 2026.0 and older",
  "provider": "Example Inc.",
  "domains": [
      "example.com",
      "example.net",
      "example.org"
    ],
  "servers": [
      {
        "type": "imap",
        "name": "imap.example.com"
      },
      {
        "type": "smtp",
        "name": "smtp.example.com"
      },
      {
        "type": "caldav",
        "port": 443,
        "url": "https://www.example.net/SOGo/dav/%EMAILADDRESS%/Calendar/personal/"
```

```
      },
      {
        "type": "carddav",
        "port": 443,
        "url": "https://www.example.net/SOGo/dav/%EMAILADDRESS%/Contacts/personal/"
      }
    ]
}
```

Once you have populated the database automx2 is ready to run.

### 7.5.4. MySQL

Step 1: Create a database.

```
CREATE DATABASE `automx2` COLLATE 'utf8mb4_general_ci';
```

Step 2: Set the database URI in your automx2 configuration. The following example uses *pymysql* as a DB driver, which is not included in the automx2 distribution.

```
[automx2]
db_uri = mysql+pymysql://user:pass@dbhost/automx2?charset=utf8mb4
```

Step 3: Launch automx2 and access the DB initialisation URL:

```
curl http://127.0.0.1:4243/initdb/
```

### 7.5.5. PostgreSQL

Step 1: Create a database.

```
CREATE DATABASE automx2 LOCALE 'en_US.utf8';
```

Step 2: Set the database URI in your automx2 configuration. The following example uses *psycopg2* as a DB driver, which is not included in the automx2 distribution.

```
[automx2]
db_uri = postgresql+psycopg2://user:pass@dbhost/automx2
```

Step 3: Launch automx2 and access the DB initialisation URL:

```
curl http://127.0.0.1:4243/initdb/
```

# 8. LDAP support

automx2 supports looking up user account data using LDAP. This is typically used to find users' login IDs for IMAP/SMTP authentication given an associated email address. Note that this is an optional configuration element commonly used by larger organisations. For smaller user bases, using placeholders may be sufficient.

The following partial LDIF snippet shows how a mail account can be defined in a widely used LDAP schema:

```
dn: uid=jdoe,ou=mailusers,dc=example,dc=com
objectClass: inetOrgPerson
objectClass: organizationalPerson
objectClass: person
objectClass: posixAccount
objectClass: top
cn: John Doe
givenName: John
homeDirectory: /var/maildata/jdoe
mail: johndoe@example.com
sn: Doe
uid: jdoe
uidNumber: 4321
[... more attributes here ...]
```

In order to allow automx2 to connect, an entry similar to the following needs to be created in the database:

```
INSERT INTO ldapserver (
    id, name, port, use_ssl,
    search_base, search_filter, attr_uid, attr_cn,
    bind_password, bind_user
) VALUES (
    100, 'ldap.example.com', 636, 1,
    'ou=mailusers,dc=example,dc=com', '(mail={0})', 'uid', 'cn',
    'PASSWORD', 'cn=automx2,ou=services,dc=example,dc=com'
);
```

An encrypted connection (LDAPS) is used and the filter and attribute names are set according to the LDIF above. It is assumed that `cn=automx2,ou=services,dc=example,dc=com` with the given password is permitted read-only access to the necessary LDAP records/attributes. The search filter needs to contain the placeholder `{0}` which will be replaced with the email address used as the lookup key.

Values of the LDAP attributes specified using `attr_uid` and `attr_cn` will be returned as the resulting login ID and common name, respectively. If your users are required to log in using their email address instead of a technical user ID, and based on the example LDIF above, set `attr_uid` to *mail* instead of *uid*:

```
UPDATE ldapserver SET attr_uid='mail' WHERE id=100;
```

Now all that is left is to connect the example.com domain to LDAP server ID 100:

```
UPDATE domain SET ldapserver_id=100 WHERE name='example.com';
```

# 9. Running automx2

Running automx2 requires to start automx2 as service and serve its output via a web server to the public. You should not run automx2 with superuser privileges. Use a dedicated user instead.

The following examples assume you have created a user and group `automx2` and have granted appropriate rights to this user:

- Read permissions for the `automx2.conf` configuration file.
- Read and access permissions for the virtual Python environment.
- Read and access permissions for the SQLite database.

Please note that the provided files are examples only, and will probably need some changes to match your particular environment. Paths for Python, Flask, databases and runtime data can vary considerably.

## 9.1. As a OpenRC service

The following is an example for a OpenRC run script `/etc/init.d/automx2` which I use for Gentoo Linux:

```
#!/sbin/openrc-run
# /etc/init.d/automx2
# Change paths in this file as necessary, depending on your
# chosen method of installation.

: ${AUTOMX2_CONF:="/etc/${RC_SVCNAME}.conf"}
: ${AUTOMX2_USER:="automx2"}
: ${AUTOMX2_ARGS:="--port 4243"}

command="/usr/bin/python"
command_args="/usr/bin/flask run ${AUTOMX2_ARGS}"
command_background="true"
command_user="${AUTOMX2_USER}"
pidfile="/run/${RC_SVCNAME}.pid"
required_files="${AUTOMX2_CONF}"

depend() {
    use logger net
```

```
    before nginx
}

start_pre() {
    export AUTOMX2_CONF
    export EPYTHON="python3.9"
    export FLASK_APP="automx2.server:app"
    export FLASK_ENV="production"
}
```

If you wish to override settings, you can copy the following to `/etc/conf.d/automx2` and uncomment/change variables according to your needs.

```
# /etc/conf.d/automx2

# Additional parameters passed to Flask
#AUTOMX2_ARGS="--host 127.0.0.1 --port 4243"

# Configuration file
#AUTOMX2_CONF="/etc/automx2.conf"

# Process owner (choose a non-privileged user)
#AUTOMX2_USER="automx2"
```

## 9.2. As a systemd service

If your system uses *systemd* you may want to deploy the following automx2.service unit file from the contrib section and place it in `/etc/systemd/system/automx2.service`. Starting with version 2025.1, automx2 will auto-detect the presence of the `NOTIFY_SOCKET` environment variable and signal service readiness to systemd.

```
[Unit]
After=network.target
Description=MUA configuration service
Documentation=https://rseichter.github.io/automx2/

[Service]
Environment=FLASK_APP=automx2.server:app
Environment=FLASK_CONFIG=production
# Change paths in this file as necessary, depending on your
# chosen method of installation.
ExecStart=/srv/www/automx2/.venv/bin/flask run --host=127.0.0.1 --port=4243
NotifyAccess=exec
Restart=always
Type=notify
User=automx2
WorkingDirectory=/var/lib/automx2
```

```
[Install]
WantedBy=multi-user.target
```

Once you have installed the service you need to tell systemd to reload its list of available services:

```
sudo systemctl daemon-reload
```

It should now be able to tell you about a service named automx2:

```
sudo systemctl status automx2
● automx2.service - MUA configuration service
     Loaded: loaded (/etc/systemd/system/automx2.service; disabled; vendor preset:
enabled)
     Active: inactive (dead)
```

Next enable and start automx2 using the following command:

```
sudo systemctl enable automx2 --now
Created symlink /etc/systemd/system/multi-user.target.wants/automx2.service →
/etc/systemd/system/automx2.service.
```

You should see automx2 enabled and running:

```
sudo systemctl status automx2
● automx2.service - MUA configuration service
     Loaded: loaded (/etc/systemd/system/automx2.service; enabled; vendor preset:
enabled)
     Active: active (running) since Mon 2021-03-01 12:54:31 CET; 19s ago
   Main PID: 126966 (python)
      Tasks: 1 (limit: 4620)
     Memory: 46.1M
     CGroup: /system.slice/automx2.service
             └─126966 /srv/www/automx2/.venv/bin/flask run --host=127.0.0.1 --port
=4243
[...]
Mar 01 12:54:32 mail python[126966]: Reading /etc/automx2.conf
Mar 01 12:54:32 mail python[126966]: Config.get: loglevel = WARNING
Mar 01 12:54:32 mail python[126966]:  * Running on http://127.0.0.1:4243/ (Press
CTRL+C to quit)
```

You are now ready to start testing automx2, as described below.

## 9.3. Manually from a shell

While logged in as an unprivileged user, change into the installation directory and start the

`.venv/bin/flask` launch script:

```
cd /srv/www/automx2
FLASK_APP=automx2.server:app .venv/bin/flask run --host=127.0.0.1 --port=4243
```

> *Handling terminal output*
>
> The launch script will deliberately keep automx2 running in the foreground, and log data will be displayed in the terminal. If you press Ctrl-C or close the shell session, the application will terminate. To run automx2 in the background, you can use a window manager like GNU Screen [https://www.gnu.org/software/screen/] or tmux [https://en.wikipedia.org/wiki/Tmux].

Now that automx2 is up and running, you need to configure the web server proxy that will receive requests from the outside and forwards them to automx2.

# 10. Testing automx2 locally

You can use *curl* in a command shell to send a GET request to your local automx2-instance. The following example assumes your service runs on localhost on port 4243. The exact output depends on your database content, but should look similar.

```
curl 'http://127.0.0.1:4243/mail/config-v1.1.xml?emailaddress=user@example.com'
```

```
<clientConfig version="1.1">
    <emailProvider id="automx2-100">
        <identity/>
        <domain>example.com</domain>
        <displayName>Example Inc.</displayName>
        <displayShortName>Example</displayShortName>
        <incomingServer type="imap">
            <hostname>mail.example.com</hostname>
            <port>993</port>
            <socketType>SSL</socketType>
            <username>%EMAILADDRESS%</username>
            <authentication>plain</authentication>
        </incomingServer>
        <incomingServer type="pop3">
            <hostname>mail.example.com</hostname>
            <port>110</port>
            <socketType>STARTTLS</socketType>
            <username>%EMAILADDRESS%</username>
            <authentication>plain</authentication>
        </incomingServer>
        <outgoingServer type="smtp">
            <hostname>mail.example.com</hostname>
            <port>587</port>
```

```
            <socketType>STARTTLS</socketType>
            <username>%EMAILADDRESS%</username>
            <authentication>plain</authentication>
        </outgoingServer>
        <!-- ... -->
    </emailProvider>
</clientConfig>
```

Having verified that automx2 returns configuration data, you should make the service available using a web server as a proxy.

# 11. Configuring a web server

While it is technically possible to run automx2 without a web server in front of it, I do not recommend doing that in a production environment. A web server can provide features automx2 was designed not to have. Features such as transport layer encryption for HTTPS (required for Mobileconfig) or, for example, the capability to rate-limit clients are handled very well by full-fledged web servers working as reverse proxies. It would be a waste to re-implement all this in a web service.

This section will explain how to configure a web server as a reverse proxy in front of automx2. Before you set up the proxy you need to tell automx2 it operates behind one. Add the `proxy_count` parameter to your automx2 configuration file or uncomment the parameter if it is already there:

```
[automx2]
# A typical production setup would use loglevel = WARNING
loglevel = WARNING

# Disable SQL command echo.  ①
db_echo = no

# SQLite database in a UNIX-like file system
db_uri = sqlite:////var/lib/automx2/db.sqlite

# Number of proxy servers between automx2 and the client (default: 0).
# If your logs only show 127.0.0.1 or ::1 as the source IP for incoming
# connections, proxy_count probably needs to be changed.  ②
proxy_count = 1
```

① Echoing SQL commands is only meant for debugging purposes.

② Set the number to reflect the number of proxies chained in front of automx2, i.e. the number of "proxy hops" a client's request must pass before it reaches automx2.

## 11.1. NGINX

The following example defines an HTTP server, which will listen for requests to both *autoconfig.example.com* and *autodiscover.example.com*. All requests will be forwarded to automx2, which listens on TCP port 4243 in this example. Requests to `/initdb` are restricted to clients connecting

from the local host. The `proxy_set_header` directives will cause NGINX to pass relevant data about incoming requests' origins.

```
# NGINX example configuration snippet to forward incoming requests to automx2.
# vim: ts=4 sw=4 et ft=nginx

http {
    server {
        listen *:80;
        listen [::]:80;
        server_name autoconfig.example.com autodiscover.example.com;
        location /initdb {
            # Limit access to clients connecting from localhost
            allow 127.0.0.1;
            deny all;
        }
        location / {
            # Forward all traffic to local automx2 service
            proxy_pass http://127.0.0.1:4243/;
            proxy_set_header Host $host;
            # Set config parameter proxy_count=1 to have automx2 process these headers
            proxy_set_header X-Forwarded-Proto http;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

## 11.2. Apache

The following example shows an Apache configuration similar to the one above. `ProxyPreserveHost` directives will cause apache to pass relevant data about incoming requests' origins.

```
# Apache 2.4 example configuration snippet to forward incoming requests to automx2.
# vim: ts=4 sw=4 et ft=apache

<VirtualHost *:80>
    ServerName autoconfig.example.com
    ServerAlias autodiscover.example.com
    ProxyPreserveHost On
    ProxyPass "/" "http://127.0.0.1:4243/"
    ProxyPassReverse "/" "http://127.0.0.1:4243/"
    <Location /initdb>
        # Limit access to clients connecting from localhost
        Order Deny,Allow
        Deny from all
        Allow from 127.0.0.1
    </Location>
```

```
</VirtualHost>
```

# 12. Sponsorship

If you are interested in sponsoring a specific feature, please contact me using the email address <*automx2 AT seichter DOT de*>.